

PROCESS MANAGEMENT

Condensed Study Notes

Generated: 2026-05-25 00:06 UTC

PROCESSMANAGEMENT

Imagine your computer is like a busy office, and each task you ask it to do is a 'project' that needs to be managed. These projects are called 'processes'.

What is a Process?

A process is basically a program that's currently running. Think of it like a chef in a kitchen. The recipe is the program (the instructions), and the chef is the computer's processor (the part that actually does the work). When the chef is actively cooking, that's the process.

Key ideas:

- A 'process' is a program that is actively being executed.
- The computer's 'processor' is like the chef, carrying out the instructions.
- Each process needs its own set of resources, like a dedicated workspace for the chef.

PROCESSCONCEPT

You know how a program is like a recipe, right? Well, when that recipe is actually being followed, with someone in the kitchen actively chopping, mixing, and cooking, that's what we call a 'process'.

So, a **process** is simply a program that is currently running. Think of it as a recipe that's being actively prepared by a chef.

In the past, especially with older computer systems that handled tasks in big batches, they used to talk about 'jobs'. A 'job' was basically a set of tasks to be done. You might still hear the term 'job' used today, and often, people use 'job' and 'process' interchangeably, like they mean the same thing. It's like calling the chef's work an 'active recipe' or a 'cooking job' – both describe what's happening!

THEPROCESS

We've learned that a 'process' is like a program that's actively running, similar to a chef busy with a recipe. Now, let's think about how a running program (a process) uses its own dedicated memory space.

Imagine a process's memory as a workspace divided into different zones, each with a specific job:

- **The Text Section:** This is where the actual instructions of your program live. Think of it like the printed recipe book itself, containing all the steps the chef needs to follow. When you launch a program, its compiled code is loaded from storage into this section.
- **The Data Section:** This zone holds information that the program needs throughout its run, like global and static variables. These are like the common ingredients that are always available in the kitchen, ready to be used by any part of the recipe. This data is set up before the main part of the program even

starts.

- **The Heap:** This is a flexible area for memory that the program can request as it's running. Imagine this like a pantry where the chef can ask for extra ingredients (memory) as needed for specific, on-the-fly tasks. When a program needs more space for something that wasn't planned from the start, it asks for it from the heap, often using commands like 'new' or 'malloc'.

- **The Stack:** This area is used for temporary storage, especially for things like local variables that only exist while a specific part of the program (like a function or a specific step in the recipe) is active. It's also used for function return values. Think of it as a small, temporary notepad the chef uses to jot down notes for a particular step, which they then discard once that step is done. Space on the stack is reserved when these variables are created and freed up when they are no longer needed.

An interesting point is that the heap and the stack start at opposite ends of the process's available free space and grow towards each other. If they ever meet, it means the process has run out of memory, leading to either a 'stack overflow error' (if the stack runs out of room) or a failed request for memory from the heap.

When a process is temporarily paused and then brought back later, the computer needs to save not just its memory, but also crucial details about where it was. Two key pieces of information saved are:

- **The Program Counter:** This is like a bookmark that remembers exactly which instruction the process was about to execute next.
- **Program Registers:** These are like tiny, super-fast storage spots within the computer's processor (the part that does the actual computing) that hold important, currently-used data. Saving their values ensures the process can pick up exactly where it left off.

PROCESS CREATION

When you run a program, a 'process' is born. Think of it like a chef starting to cook a specific dish. The process is the active program, just like the chef is the active cook.

There are a few main ways a new process gets created:

- When the computer first starts up (system initialization).
- When a program that's already running asks the operating system to create a new one.
- When you, the user, tell the computer to start something new.
- When a batch job (a set of tasks to run automatically) is kicked off.

When a process is created, it gets a unique number called a 'process identifier' or 'PID'. This is like a ticket number for that specific running program. Each process also keeps track of the PID of the process that created it, which is called its 'parent process identifier' or 'PPID'.

Processes can even create other processes! The process that does the creating is called the 'parent', and the one that gets created is called the 'child'. This is a bit like a parent chef teaching their apprentice chef how to make a new dish.

On systems like UNIX, there's a special process called the 'scheduler' (often named 'sched') with PID 0. When the system starts, it launches another process called 'init' (with PID 1). 'init' is a very important

process because it goes on to start up all the other system services and user logins, making it the 'ultimate parent' of almost every other process on the system. You can imagine this like a head chef overseeing the entire kitchen, with other chefs (processes) reporting up to them.

(A 'system call' is a way for a running program to ask the operating system to do something for it, like creating a new process. It's the bridge between your program and the computer's core services.)

PROCESSTERMINATION

Just like a chef finishes a recipe or a task, a running program, called a 'process', eventually stops. This stopping is called 'termination'.

There are a few main ways a process can end:

1. **Normal Exit (Voluntary):** This is when a process successfully completes its job. Imagine a compiler finishing its work of turning your code into a runnable program. It then tells the operating system (the computer's manager) that it's done, using a special command like `exit ( )` (in systems like UNIX) or `ExitProcess` (in Windows).

2. **Error Exit (Voluntary):** Sometimes, a process realizes it has run into a problem it can't fix. For example, if you try to compile a program but the file it needs doesn't exist, the compiler will announce the error and then stop itself.

3. **Fatal Error (Involuntary):** This happens when a process makes a mistake that's so bad it can't continue. This is often due to a bug in the program itself, like trying to do something impossible, such as dividing by zero, or trying to execute an instruction the computer doesn't understand.

4. **Killed by Another Process (Involuntary):** One process can be instructed to terminate another process. This is like one chef telling another to stop what they're doing. In UNIX, this is done with a `KILL` command, and in Windows, it's called `TerminateProcess`.

Other reasons a process might be terminated by the system:

- A 'parent' process (which created other processes, called 'children') might decide it no longer needs its child processes and terminate them.
- If a parent process exits, the system might decide to let its child processes continue running, or it might terminate them too.
- A process might be terminated if the system receives a command to do so, like a `KILL` command, or if it encounters an unhandled interruption.
- The system might run out of essential resources needed by the process.

What happens when a process terminates?

When a process finishes, all the resources it was using are returned to the system. This includes things like closing any files it had open. The process also usually sends back a status code to its parent process, indicating whether it finished successfully (often a zero code) or encountered problems (a non-zero code).

Orphaned Processes and Zombies:

Sometimes, a child process might be left without a parent (if the parent process exited first). These are called 'orphaned processes'. In many systems, a special process called 'init' takes care of these orphans and eventually terminates them.

There's also a state called a 'zombie' process. This is a process that has finished its work but is still lingering because its parent hasn't yet acknowledged its termination. These zombie processes are eventually cleaned up by being inherited by 'init' and then killed off.

PROCESS STATE

Imagine a busy kitchen where different chefs are working on different recipes. Each chef (or process) has a specific stage they are in for their current task.

As a program runs, it goes through different phases, and we call these phases its 'state'. The state tells us what the process is currently doing.

Here are the main states a process can be in:

- **New:** This is like a new order just coming into the kitchen. The process is in the very beginning stages of being created.
- **Running:** This means the main worker (the CPU – the Central Processing Unit, which is like the computer's brain that does all the work) is actively following the process's instructions right now.
- **Ready:** Think of this as a chef who has all their ingredients ready and is just waiting for their turn to use the main cooking station (the CPU). The process has everything it needs to run, but the CPU is busy with another process at the moment. It might get interrupted from the 'Running' state and moved here.
- **Waiting:** This is like a chef who has to stop because they are waiting for something specific to happen. For example, they might be waiting for a delivery of ingredients (like keyboard input or data from a disk), or waiting for another chef to finish a part of their task before they can continue.
- **Terminated:** This is when a process has finished its job, like a chef completing a dish. It's done and is waiting to be cleaned up and removed from the system.

Diagram of process state

Imagine you're managing a busy library. Each book (program) needs a special card to keep track of its status and where it is. In a computer, each running program, which we call a 'process', also has a special card managed by the computer's main operator (the operating system). This card is called a **Process Control Block (PCB)**.

The PCB is like a detailed file folder for each process. It holds all the essential information the operating system needs to keep track of that running program.

Here's what's inside a typical PCB:

- **Process ID (PID):** This is like a unique library card number for each process, making it easy to identify. It also includes the **Parent Process ID**, which is like knowing who 'checked out' this process originally.
- **Process State:** This tells us what the process is currently doing – is it actively running, waiting for something, or just created? (We talked about these states before!).
- **Program Counter:** Think of this as a bookmark in the process's instruction book, showing the very next instruction the computer needs to execute. This bookmark needs to be saved if the process is paused.
- **CPU Registers:** These are like the scratchpads and temporary storage the computer's 'brain' (the CPU) uses to do its work quickly. When a process is actively running on the CPU, its current work is stored in these registers. Different computer designs have different numbers and types of registers.
- **Memory Management Information:** This is like the library's catalog system for the process's 'bookshelf'. It includes details about how much memory the process is using and how it's organized, such as **page tables** or **segment tables** (ways to map the process's view of memory to the actual physical memory).
- **I/O Status Information:** This is like a list of all the library services the process is currently using, such as which devices (like printers or disks) are allocated to it and which 'books' (files) are open.
- **Accounting Information:** This tracks how much 'time' (CPU time) the process has used, who is responsible for it (user account), and any limits placed on its usage.
- **CPU-Scheduling Information:** This includes details like the process's 'priority' (how important it is) and where it's waiting in line for its turn on the CPU (pointers to scheduling queues).

It's important to remember that the exact contents of a PCB can vary depending on the specific operating system, just like different libraries might have slightly different ways of organizing their card catalogs.

Process control block(PCB) OPERATIONSONPROCESSES

Now that we know what a Process Control Block (PCB) is – that handy information card for each running program – let's look at what the operating system can do with these PCBs to manage processes.

Think of the operating system as a very organized librarian, and the PCBs are like the library's index cards for each book (process) that's currently checked out and being read. The librarian can perform certain actions based on the information on these cards.

Here are some key operations the operating system can perform on processes, using their PCBs:

- **Process Creation:** This is like the librarian getting a brand new book and creating its index card for the first time. The operating system creates a new process when a program is run. This involves setting up a new PCB for it, which holds all the initial information about this new running program.
- **Process Scheduling or Dispatching:** Imagine the librarian deciding which reader gets to use a popular book next. Process scheduling is how the operating system decides which process gets to use the CPU (the computer's processor, which does the actual work) at any given moment. It uses the information in the PCBs to make these decisions, making sure all processes get a fair share of the CPU's time.

A tree of processes on a typical Linux system

Imagine you have a main project manager (the parent process) who needs to delegate some tasks. When the manager creates a new worker (a child process), they might give the worker some of the same tools and resources they have, but perhaps not all of them. This is to make sure that even if a worker gets a bit carried away, they can't use up *all* the company's resources.

After creating a new worker, the main project manager has a couple of choices on how to proceed:

1. **Wait it out:** The manager can stop what they're doing and wait until the worker finishes their assigned task. This is like a manager waiting for a report from an employee before continuing with their own work. In systems like UNIX, this is done using a command called `wait ( )`. The manager might wait for a specific worker or for any worker to finish.
2. **Keep going:** The manager can let the worker start their task and continue with their own work at the same time. This is what happens when you tell your computer to run a program in the background – your main command window doesn't freeze. Sometimes, the manager might start a few workers, do a little bit of their own work, and then wait for the workers to finish later. This is like a manager delegating multiple tasks, doing some independent work, and then checking in on the delegated tasks.

Preemption

Imagine you're in a busy kitchen, and the head chef is currently working on a very important dish. Suddenly, an urgent order comes in for a VIP guest that needs to be made *right now*. The head chef might pause what they're doing with the first dish, put it aside, and immediately start on the VIP order because it's more important.

In the world of computers, 'preemption' is a similar idea. It's the ability of the operating system to interrupt a task that is currently running, especially if a more important task needs attention.

- **Preemption:** This is the act of stopping a currently running task so that another task can take over. Think of it like a referee blowing a whistle to pause one game so a more critical match can begin.
- **Scheduled task:** This refers to any program or part of a program that the operating system has decided is ready to run and is currently using a resource, like the computer's processor.
- **Higher priority task:** This is a task that the operating system considers more important or urgent than the one currently running. It needs to be dealt with sooner.
- **Processor:** This is the part of the computer that actually does the thinking and calculations, like the brain of the computer.
- **I/O:** This stands for Input/Output, and it refers to tasks where the computer is communicating with the outside world, like reading from a hard drive or sending information to a screen. These can also be resources that a task might be waiting for or using.

So, preemption means the operating system can take away the processor or another resource from a task that's currently using it, if another task with a higher priority needs that resource urgently.

Process Scheduling

Imagine a busy office where many people are waiting to use the only available photocopier. Scheduling is like the office manager deciding who gets to use the photocopier next.

In computer terms, 'scheduling' or 'dispatching' is the action the operating system (OS) takes to move a program that's ready to run from a waiting list to the actual 'running' state, where it gets to use the computer's main brain (the CPU).

This happens when:

- There are free resources (like the CPU becoming available).
- A program that's considered more important than the one currently running needs to take its turn.

Blocking

Imagine a busy kitchen where a chef is trying to make a complex meal. Sometimes, the chef needs a specific ingredient that isn't ready yet, or another chef is using a tool they need. In this situation, the first chef has to pause their work and wait until the ingredient is ready or the tool is free. This pausing to wait is like 'blocking' in a computer system.

When a program (we call a running program a 'process') is 'blocked', it means the system makes it stop running for a while. This happens when the process needs to use something that is currently being used by another process. These shared things are called 'shared resources'.

Think of 'shared resources' as things like:

- The CPU: The main 'brain' that does the work. Only one process can actively use it at a time.
- Network or network interfaces: Like the phone line or internet connection that multiple devices might need to share.
- Memory: The workspace where programs store their information.
- Disk: Where files are stored long-term.

So, when a process needs one of these shared resources and it's already in use by another process, the system puts the waiting process into a 'waiting state'. It can't do anything else until the resource it needs becomes available again.

INTER-PROCESS COMMUNICATION

Imagine you have different programs running on your computer at the same time, like a web browser, a music player, and a word processor. Sometimes, these programs might need to talk to each other or share information.

Processes can either be:

- **Independent Processes:** These are like separate individuals who don't need to interact or share anything. What one does doesn't affect the other, and vice versa.
- **Cooperating Processes:** These are like team members who need to work together. They can affect each other, or be affected by each other. This usually happens when they need to share data.

Why would programs need to cooperate?

- **Information Sharing:** Think about multiple people needing to read from the same document or access the same shared calendar. In computers, this could be several programs needing to access the same file.
- **Convenience:** You might be doing several things at once – typing an email, listening to music, and maybe even running a small test program. These are all separate tasks (processes) that you're managing.
- **Modularity:** Sometimes, a big complex system is built by breaking it down into smaller, cooperating parts. Like a database system where one part handles requests (the client) and another part stores the information (the server).
- **Computation Speedup:** If a big problem can be split into smaller pieces, different parts of your computer (especially if it has multiple processors, which are like multiple brains) can work on those pieces at the same time, solving the problem much faster.

When programs need to cooperate, they need a way to communicate. The two main ways this happens are:

1. Shared Memory:

- **Analogy:** Imagine a shared whiteboard where multiple people can write and read information. Once the whiteboard is set up, everyone can quickly see and add things without needing a special messenger.
- **Technical Detail:** With shared memory, one process can set aside a portion of its memory that other processes can access directly. This is very fast because the operating system doesn't need to get involved for every piece of information shared. However, it can be trickier to set up correctly and doesn't work as well if the processes are on different computers. It's best for sharing large amounts of data quickly on the same computer.

2. Message Passing:

- **Analogy:** Think of sending letters or text messages. Each time you want to send information, you have to package it up, send it, and the other person has to receive and open it. This takes more steps than writing on a shared whiteboard.
- **Technical Detail:** With message passing, processes communicate by sending and receiving messages to each other. This requires the operating system to help with every message transfer, making it slower than shared memory. However, it's generally simpler to set up and works well for communication across different computers. It's a good choice when you don't need to transfer huge amounts of data very often, or when computers are networked together.